# Fuzz Testing for Input Validation Bugs

Hypothesis-based fault discovery, defensive fixes and regression verification


SUBMITTED BY
Aayaan Kamil

STUDENT REGISTRATION NUMBER
PES2UG24CS017

LAB
07


**03** Functions examined
Seat | Price | Rating

**03** Tests failed before fixes
8 distinct failures reported

**03** Tests passed after fixes
0.85 s final run

## METHOD / FOUR REQUIRED EXECUTION STAGES

**01** Original run

**02** Failure discovery

**03** Fixed run

**04** Passing re-test

## RESULTS AT A GLANCE

| | | |
|---|---|---|
| `parse_seat` | Malformed / non-text seat input | Return normalized seat or None |
| `total_price` | Invalid CSV values or input types | Apply 18% GST to valid nonnegative values; else 0.0 |
| `average_rating` | Empty / invalid / out-of-range ratings | Average valid 1-5 ratings; else 0.0 |

# Run the original application

Step 2 / Baseline execution of the assigned Movie Tickets script

CAPTURE 01 | ORIGINAL TERMINAL OUTPUT

```
PS C:\Users\aayaa\Downloads\L6_L7_15_movie_tickets\Lab 7> python movie_tickets.py
==== Running parse_seat ====
Enter a seat number (e.g. H12): H12
Row and Seat: ('H', 12)

==== Running total_price ====
Enter ticket prices as CSV (e.g. 150,250,400): 150,250,400
Total with GST: 944.0

==== Running average_rating ====
Enter ratings separated by spaces (e.g. 4 5 3): 4 5 3
Average Rating: 4.0
```

## Observed baseline

Command: python movie_tickets.py

Seat H12 returned ('H', 12); prices 150,250,400 returned 944.0 including 18% GST.

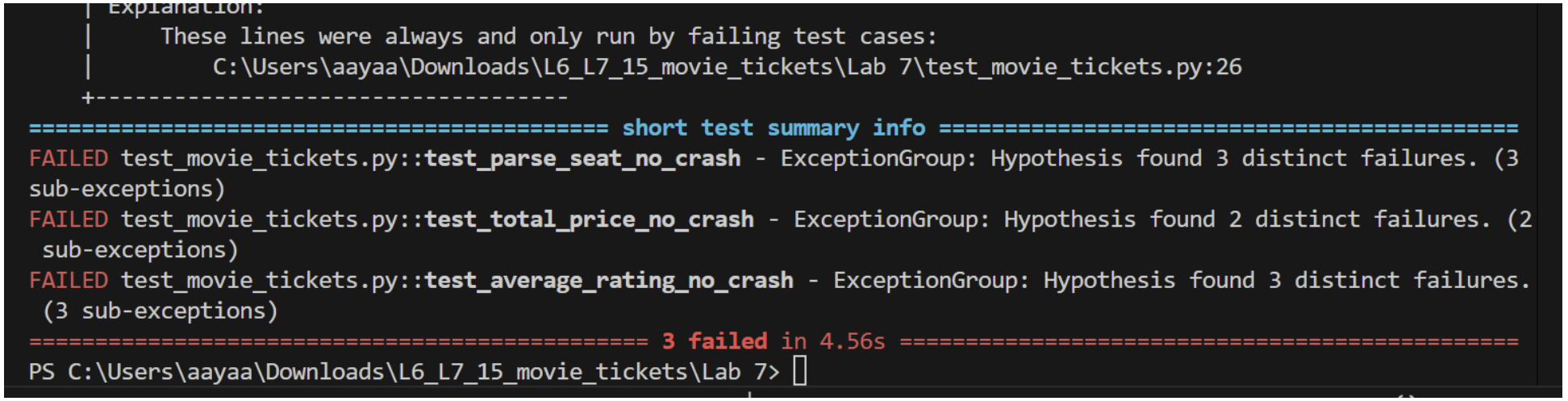
Ratings 4 5 3 returned an average of 4.0. These valid inputs establish the initial behavior.

**WHY THIS MATTERS** A passing normal-input run does not establish resilience to malformed inputs.

# Expose failures with Hypothesis

Steps 3-4 / Complete the property tests and run them against the buggy code

CAPTURE 02 | FAILING PYTEST / HYPOTHESIS RUN




## Failure summary

3 TESTS FAILED

8 DISTINCT FAILURES

Command: python -m pytest test_movie_tickets.py -v

Hypothesis reported 3 distinct failures for parse_seat, 2 for total_price, and 3 for average_rating.

This establishes a reproducible failing baseline before the application code was corrected.

# Apply defensive input validation

Step 5 / Run the updated application after hardening all three functions

CAPTURE 03 | CORRECTED APPLICATION OUTPUT

```
PS C:\Users\aayaa\Downloads\L6_L7_15_movie_tickets\Lab 7> python movie_tickets.py
==== Running parse_seat ====
Enter a seat number (e.g. H12): h13
Row and Seat: ('H', 13)

==== Running total_price ====
Enter ticket prices as CSV (e.g. 150,250,400): 130,230,330
Total with GST: 814.2

==== Running average_rating ====
Enter ratings separated by spaces (e.g. 4 5 3): 4 5 6
Average Rating: 0.0
PS C:\Users\aayaa\Downloads\L6_L7_15_movie_tickets\Lab 7>
```

## Behavior seen in the corrected run

| INPUT | OBSERVED RESULT | VALIDATION NOTE |
|---|---|---|
| `Seat: h13` | `('H', 13)` | Lowercase row normalized |
| `Prices: 130,230,330` | `814.2` | 18% GST applied |
| `Ratings: 4 5 6` | `0.0` | Out-of-range 6 rejected |

# Confirm the passing regression run

Step 6 / Repeat the same Hypothesis tests against the corrected program

CAPTURE 04 | FINAL HYPOTHESIS TEST RESULTS

```
n313\python.exe
cachedir: .pytest_cache
hypothesis profile 'default'
rootdir: C:\Users\aayaa\Downloads\L6_L7_15_movie_tickets\Lab 7
plugins: anyio-4.16.0, hypothesis-6.170.15, platformdirs-4.12.4
collected 3 items

test_movie_tickets.py::test_parse_seat_no_crash PASSED                   [ 33%]
test_movie_tickets.py::test_total_price_no_crash PASSED                  [ 66%]
test_movie_tickets.py::test_average_rating_no_crash PASSED               [100%]

============================================== 3 passed in 0.85s ==============================================
PS C:\Users\aayaa\Downloads\L6_L7_15_movie_tickets\Lab 7>
```

## Final verification

3 TESTS PASSED

0.85 s

test_parse_seat_no_crash | test_total_price_no_crash | test_average_rating_no_crash

All three no-crash property tests passed in the recorded final run.

The result supports the tested robustness claim; it does not prove correctness for every possible input.

SUBMISSION FILES This PDF / movie_tickets.py / test_movie_tickets.py